Practical-8

Problem Definition

Execute complex analytical aggregation queries across large datasets and inspect their internal database execution plans. The practical focuses on identifying performance bottlenecks, such as full-table scans and sorting overheads, and applying optimization techniques such as indexing, data partitioning, and query refactoring.

The implementation uses a clickstream tracking dataset containing user engagement records. Queries are executed before and after optimization to compare execution plans, query runtimes, and performance.

The practical aims to understand how database query engines process analytical workloads and how optimization techniques improve performance when working with large datasets.

Dataset

The dataset consists of simulated or prepared clickstream tracking records containing user engagement information.

The records may include:

- Event ID
- User ID
- Session ID
- Page URL
- Event type
- Event timestamp
- Device type
- Country or region
- Session duration
- Engagement metrics

The dataset is used to perform analytical operations such as counting events, calculating user activity, grouping records by event type, and measuring engagement across different regions.

Code:

```
import duckdb
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time



print("=" * 70)
print("PRACTICAL 8: QUERY PROCESSING AND OPTIMIZATION")
print("=" * 70)

# STEP 1: CONNECT TO DUCKDB

con = duckdb.connect("practical8.duckdb")

print("\n[STEP 1] DuckDB connection established successfully.")

# STEP 2: GENERATE CLICKSTREAM DATASET

print("\n[STEP 2] Generating clickstream dataset...")

np.random.seed(42)

n = 100_000

df = pd.DataFrame({
    "event_id": np.arange(1, n + 1),
```

```
    "user_id": np.random.randint(1, 10001, n),
    "session_id": np.random.randint(1, 20001, n),
    "event_type": np.random.choice(
        ["page_view", "click", "purchase", "scroll"], n
    ),
    "country": np.random.choice(
        ["India", "USA", "UK", "Canada", "Australia"], n
    ),
    "device_type": np.random.choice(
        ["Mobile", "Desktop", "Tablet"], n
    ),
    "duration_seconds": np.random.randint(1, 601, n),
    "event_timestamp": pd.to_datetime(
        np.random.randint(
            1767225600, 1798761600, n
        ),
        unit="s"
    )
})

print("Dataset generated successfully.")
print("Total records:", len(df))
print("\nDataset preview:")
print(df.head())

# STEP 3: CREATE DUCKDB TABLE

print("\n[STEP 3] Creating clickstream table...")
```

```
con.register("clickstream_df", df)

con.execute("""
    CREATE OR REPLACE TABLE clickstream AS
    SELECT * FROM clickstream_df
""")

print("Clickstream table created successfully.")

total_records = con.execute("""
    SELECT COUNT(*) FROM clickstream
""").fetchone()[0]

print("Total records in table:", total_records)

# STEP 4: BASIC ANALYTICAL AGGREGATION QUERY

print("\n[STEP 4] Executing analytical aggregation query...")

query = """
SELECT
    event_type,
    COUNT(*) AS total_events,
    AVG(duration_seconds) AS average_duration
FROM clickstream
GROUP BY event_type
ORDER BY total_events DESC
"""
```

```
result = con.execute(query).df()

print("\nAnalytical Query Result:")
print(result.to_string(index=False))

# STEP 5: EXPLAIN QUERY EXECUTION PLAN

print("\n[STEP 5] Query execution plan:")

plan = con.execute(
    "EXPLAIN " + query
).df()

print(plan.to_string(index=False))

# STEP 6: EXPLAIN ANALYZE

print("\n[STEP 6] Actual query execution analysis:")

actual_plan = con.execute(
    "EXPLAIN ANALYZE " + query
).df()

print(actual_plan.to_string(index=False))

# STEP 7: BASELINE QUERY PERFORMANCE

print("\n[STEP 7] Measuring baseline query performance...")
```

```
baseline_query = """
SELECT
    country,
    event_type,
    COUNT(*) AS total_events,
    SUM(duration_seconds) AS total_duration
FROM clickstream
WHERE country = 'India'
GROUP BY country, event_type
ORDER BY total_events DESC
"""

# Warm-up execution to reduce first-run compilation effects
con.execute(baseline_query).fetchall()

start = time.perf_counter()

baseline_result = con.execute(
    baseline_query
).df()

baseline_time = time.perf_counter() - start

print("\nBaseline Query Result:")
print(baseline_result.to_string(index=False))

print(f"\nBaseline execution time: {baseline_time:.6f} seconds")

# STEP 8: CREATE INDEX AND MEASURE PERFORMANCE
```

```
print("\n[STEP 8] Creating index on country column...")

con.execute("""
    CREATE INDEX IF NOT EXISTS idx_clickstream_country
    ON clickstream(country)
""")

print("Index created successfully.")

# Display execution plan after index creation
print("\nExecution plan after indexing:")

index_plan = con.execute(
    "EXPLAIN " + baseline_query
).df()

print(index_plan.to_string(index=False))

# Warm-up execution
con.execute(baseline_query).fetchall()

start = time.perf_counter()

optimized_result = con.execute(
    baseline_query
).df()

optimized_time = time.perf_counter() - start
```

```
print("\nQuery Result After Indexing:")
print(optimized_result.to_string(index=False))

print(
    f"\nExecution time after indexing: "
    f"{optimized_time:.6f} seconds"
)

# Verify that the baseline and indexed results are equivalent
pd.testing.assert_frame_equal(
    baseline_result,
    optimized_result
)

print("Verification successful: results match.")

# STEP 9: FILTER PUSHDOWN / FILTERED AGGREGATION

print("\n[STEP 9] Executing filtered analytical query...")

filter_pushdown_query = """
SELECT
    event_type,
    COUNT(*) AS total_events,
    AVG(duration_seconds) AS average_duration
FROM clickstream
WHERE country = 'India'
  AND device_type = 'Mobile'
```

```
GROUP BY event_type
ORDER BY total_events DESC
"""

print("\nFiltered query execution plan:")

filter_plan = con.execute(
    "EXPLAIN " + filter_pushdown_query
).df()

print(filter_plan.to_string(index=False))

# Warm-up execution
con.execute(filter_pushdown_query).fetchall()

start = time.perf_counter()

filtered_result = con.execute(
    filter_pushdown_query
).df()

filter_time = time.perf_counter() - start

print("\nFiltered Query Result:")
print(filtered_result.to_string(index=False))

print(f"\nFiltered query time: {filter_time:.6f} seconds")

# STEP 10: CTE-BASED QUERY REFACTORING
```

```
print("\n[STEP 10] Executing CTE-based analytical query...")

cte_query = """
WITH country_activity AS (
    SELECT
        country,
        event_type,
        COUNT(*) AS total_events,
        AVG(duration_seconds) AS average_duration
    FROM clickstream
    GROUP BY country, event_type
)
SELECT
    country,
    event_type,
    total_events,
    average_duration
FROM country_activity
WHERE total_events > 100
ORDER BY total_events DESC
"""

print("\nCTE Query Result:")

cte_result = con.execute(cte_query).df()

print(cte_result.to_string(index=False))
```

```
print("\nCTE query execution plan:")

cte_plan = con.execute(
    "EXPLAIN ANALYZE " + cte_query
).df()

print(cte_plan.to_string(index=False))

# STEP 11: PERFORMANCE COMPARISON

print("\n[STEP 11] Comparing query execution times...")

performance = pd.DataFrame({
    "Query": [
        "Baseline",
        "After Indexing",
        "Filter Pushdown"
    ],
    "Execution Time (seconds)": [
        baseline_time,
        optimized_time,
        filter_time
    ]
})

print("\nPerformance Comparison Table:")
print(performance.to_string(index=False))

# Calculate the observed difference between baseline and indexed runs
```

```
if baseline_time > 0:
    difference_percent = (
        (baseline_time - optimized_time)
        / baseline_time
    ) * 100

    print(
        "\nObserved runtime change after indexing: "
        f"{difference_percent:.2f}%"
    )

    if difference_percent > 0:
        print("The indexed run was faster in this measurement.")
    elif difference_percent < 0:
        print("The indexed run was slower in this measurement.")
    else:
        print("Both measured runtimes were equal.")

# STEP 12: VISUALIZE PERFORMANCE

print("\n[STEP 12] Generating performance comparison chart...")

plt.figure(figsize=(9, 5))

bars = plt.bar(
    performance["Query"],
    performance["Execution Time (seconds)"]
)
```

```
plt.xlabel("Query / Optimization Technique")
plt.ylabel("Execution Time (seconds)")
plt.title("Practical 8: Query Performance Comparison")
plt.xticks(rotation=15)
plt.grid(axis="y", linestyle="--", alpha=0.4)

for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height:.5f}s",
        ha="center",
        va="bottom"
    )

plt.tight_layout()
plt.savefig("practical8_performance.png", dpi=300)
plt.show()

print("Performance chart saved as practical8_performance.png")

# STEP 13: FINAL VERIFICATION

print("\n[STEP 13] Final verification...")

print(
    "Total clickstream records:",
    con.execute(
```

```
        "SELECT COUNT(*) FROM clickstream"
    ).fetchone()[0]
)

print("\nEvent counts:")

event_counts = con.execute("""
    SELECT event_type, COUNT(*) AS total
    FROM clickstream
    GROUP BY event_type
    ORDER BY total DESC
""").df()

print(event_counts.to_string(index=False))

print("\nFinal baseline query execution plan:")

final_plan = con.execute(
    "EXPLAIN " + baseline_query
).df()

print(final_plan.to_string(index=False))

# STEP 14: CLOSE DATABASE CONNECTION

con.close()

print("\n" + "=" * 70)
print("PRACTICAL 8 COMPLETED SUCCESSFULLY")
```

```
print("Database file: practical8.duckdb")

print("Performance chart: practical8_performance.png")

print("=" * 70)
```

Key Questions / Analysis / Interpretation

Question 1: Explain what a full-table disk scan operation means within an execution plan, and describe its computational impact on large datasets.

Answer:

A full-table scan is a database operation in which the query engine examines the rows of a table to find records that satisfy the query conditions.

For example:

```
SELECT *

FROM clickstream

WHERE country = 'India';
```

If the database uses a full-table scan, it examines the table's records to identify those belonging to India.

The main impacts include:

- Higher I/O consumption: More data may need to be read from storage.
- Increased execution time: Large tables can take longer to scan.
- CPU overhead: The engine must evaluate the filtering conditions.
- Memory pressure: Additional resources may be required for sorting and aggregation.
- Scalability challenges: Query costs can grow as the dataset increases.

A full-table scan is not always inefficient. It can be the best option when most of a table's records must be processed. Indexes and suitable partitioning can improve queries that access only a small portion of the data.

Question 2: How does a columnar storage layout fundamentally change query processing and optimization compared to a traditional row-oriented layout?

Answer:

Row-oriented storage stores the values of a record together, whereas columnar storage stores values from the same column together.

For example, a row-oriented layout stores each clickstream event as a complete record. A columnar layout stores the country values together, the event_type values together, and so on.

The main differences are:

- Data access: Columnar storage can read only the columns required by a query.
- Compression: Similar values in a column often compress efficiently.
- Analytical performance: Aggregations such as SUM(), COUNT(), and AVG() can benefit from columnar processing.
- I/O efficiency: Unnecessary columns may be skipped.
- Row-level operations: Row-oriented layouts are often suitable for frequent individual record inserts and updates.

Therefore, columnar storage is particularly useful for analytical workloads involving large datasets, while row-oriented storage is commonly used for transactional workloads.

Question 3: What are the architectural costs and performance penalties associated with creating too many secondary indexes on volatile, frequently updated database tables?

Answer:

Secondary indexes help a database locate records efficiently, but each additional index requires storage and maintenance.

Creating too many indexes on frequently updated tables can cause:

- Storage overhead: Each index consumes additional disk space.
- Slower INSERT operations: New records may need to update multiple indexes.
- Slower UPDATE operations: Changes to indexed columns require index maintenance.
- Slower DELETE operations: Associated index entries must be maintained.
- Increased write amplification: Additional storage operations are required.
- Maintenance overhead: Indexes may require rebuilding or other maintenance operations.
- Query planning complexity: The optimizer must consider available access paths.

Therefore, indexes should be selected based on query patterns, filtering conditions, join requirements, and measured performance rather than creating an index on every column.

Supplementary Problem

Problem statement: Take a highly nested SQL query containing multiple subqueries, inspect its execution plan, and rewrite it using optimized Common Table Expressions (CTEs) or window functions to reduce execution costs.

Solution approach:

1. Create a complex query using multiple subqueries on the clickstream table.
2. Execute EXPLAIN ANALYZE to inspect the query plan.
3. Identify expensive operations such as repeated scans, sorting, and aggregation.
4. Rewrite the query using CTEs to improve readability and structure.
5. Use window functions when calculations are needed across related rows without collapsing them into a single grouped result.
6. Compare the original and rewritten query runtimes.
7. Verify that both queries produce equivalent results.

Example of using a CTE:

```
WITH country_activity AS (
    SELECT
        country,
        event_type,
        COUNT(*) AS total_events,
        AVG(duration_seconds) AS avg_duration
    FROM clickstream
    GROUP BY country, event_type
)
SELECT
    country,
    event_type,
    total_events,
```

```
    avg_duration
FROM country_activity
WHERE total_events > 100
ORDER BY total_events DESC;
```

Output

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8> Get-ChildItem -Recurse -Depth 2
    Directory: C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        30-09-2026     10:37                data
d-----        30-09-2026     10:38                results
d-----        30-09-2026     10:37                screenshots
d-----        30-09-2026     10:36                scripts
d-----        30-09-2026     10:35                sql
-a----        30-09-2026     10:37          10940 README.md
-a----        30-09-2026     10:37          31205 REPORT_NOTES.md
-a----        30-09-2026     10:34             78 requirements.txt
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8> python scripts/01_generate_clickstream_dataset.py
============================================================
TASK 1: MULTI-MILLION CLICKSTREAM DATASET GENERATION
============================================================
Target Row Count:     2,500,000 records
DuckDB Target File:   .../data/clickstream_analytics.duckdb
Generating synthetic clickstream logs with high cardinality...
Exporting uncompressed/Snappy baseline Parquet format...
------------------------------------------------------------
Generated Rows:       2,500,000
Distinct Users:       50,000 (High Cardinality)
Distinct Sessions:    500,000 (High Cardinality)
Distinct URLs:        5,000
Time Range:           2026-01-01 00:00:00 to 2026-03-31 23:59:59
DuckDB Storage Size:  337.01 MB
Parquet Storage Size: 133.27 MB
Generation Runtime:   31.51 seconds
Dataset Seeding:      PASS (2,500,000 Records Ready)
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8> python scripts/02_diagnose_full_table_scans.py
============================================================
TASK 2: EXECUTION PLAN PROFILING & BOTTLENECK DIAGNOSIS
============================================================
--- Scenario A: Full Scan with SELECT * (Row-store like read) ---
Wide Scan Runtime:       118.45 ms
Execution Plan Node:     SEQ_SCAN [all 13 columns materialized]
Top-K Sort Overhead:     ORDER BY response_time_ms materialized

--- Scenario B: Columnar Projection Pushdown (Only 2 columns) ---
Projected Scan Runtime:  18.20 ms
Execution Plan Node:     SEQ_SCAN [Projections: event_id, response_time_ms]
Columnar Speedup:        6.51x faster (Avoided payload/URL disk I/O)

--- Scenario C: High-Cardinality Multi-Column Aggregation ---
Aggregation Runtime:     42.10 ms across 2.5M rows
Unique Groups Generated: 108 aggregated buckets
Execution Plan Engine:   PERFECT_HASH_GROUP_BY -> TOP_N / ORDER_BY
------------------------------------------------------------
Execution Plan Profiling Status: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8> python scripts/03_indexing_and_art_trees.py
============================================================
TASK 3: SECONDARY INDEX DESIGN & ART PLAN OPTIMIZATION
============================================================
--- 1. Point Lookup Baseline (NO INDEX) on session_id ---
Target Session ID:       sess_6461613835363561393133386134633634 66...
Matched Records:         5
Execution Plan Node:     SEQ_SCAN (Scanned all 2,500,000 records)
Baseline Lookup Time:    109.20 ms

--- 2. Building Secondary Index (Adaptive Radix Tree - ART) ---
ART Index Created:       idx_clickstream_session on session_id
Index Construction Time: 23227.41 ms
Composite Index Created: idx_clickstream_user_time on (user_id, event_timestamp)

--- 3. Point Lookup Optimized (WITH ART INDEX) ---
Execution Plan Node:     INDEX_SCAN [idx_clickstream_session]
Optimized Lookup Time:   2.32 ms
Index Speedup Factor:    47.09x FASTER (Up to 116x in cold cache)
------------------------------------------------------------
Secondary Indexing Verification: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8> python scripts/04_physical_data_partitioning.py
============================================================
TASK 4: PHYSICAL DATA PARTITIONING & PARTITION PRUNING
============================================================
Exporting Hive-style partitions into: partitioned_clickstream
Partitions Created:      6 partition directories
Partition Directories:   event_type=add_to_cart, checkout, click, filter...
Partition Export Time:   4.58 seconds

--- 2. Baseline Analytical Aggregation (Unpartitioned Table) ---
Unpartitioned Plan:      SEQ_SCAN [Scanned all 2,500,000 rows across all types]
Unpartitioned Runtime:   29.88 ms

--- 3. Optimized Analytical Aggregation (With Partition Pruning) ---
Partitioned Plan:        READ_PARQUET [Hive Pruning: skipped 5 partitions]
Rows Scanned:            ~416,666 rows (Pruned 2,083,334 rows from disk)
Partitioned Runtime:     21.47 ms
Partitioning Speedup:    1.39x FASTER (Direct I/O reduction: 83.3%)
Data Parity Check:       PASS (Identical analytical metrics returned)
------------------------------------------------------------
Physical Data Partitioning Status: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8> python scripts/05_supplementary_query_refactoring.py
============================================================
TASK 5: SUPPLEMENTARY PROBLEM - NESTED SUBQUERY REFACTORING
============================================================
--- 1. Baseline: Unoptimized Multi-Level Nested Subqueries ---
Nested Subquery Plan:    Multiple SEQ_SCANs, Correlated Subplans, Nested Loops
Nested Subquery Rows:    25 rows returned
Nested Subquery Runtime: 18110.07 ms

--- 2. Optimized: Single-Pass CTEs + Window Functions ---
Optimized Plan:          Single SEQ_SCAN -> HASH_GROUP_BY -> WINDOW Streaming
Optimized Query Rows:    25 rows returned
Optimized Query Runtime: 531.81 ms
Refactoring Speedup:     34.05x FASTER
------------------------------------------------------------
Query Plan Optimization Breakdown:
• Baseline: 4 redundant full table scans + correlated subquery joins.
• Optimized: 1 single table scan + streaming window evaluation.
• Parity Check: Identical top analytical records produced.
Supplementary Query Refactoring: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8> python scripts/06_secondary_index_write_penalty.py
============================================================
TASK 6: SECONDARY INDEX WRITE PENALTY & VOLATILITY BENCHMARK
============================================================
Batch INSERT Volume:     100,000 records per test configuration

[0 Secondary Indexes (Baseline)]
  Indexes Active:        0 | Duration: 345.28 ms | Rate: 289,616 rows/sec
  Write Penalty:         1.00x relative to baseline

[1 Secondary Index (session_id)]
  Indexes Active:        1 | Duration: 1102.06 ms | Rate: 90,739 rows/sec
  Write Penalty:         3.19x relative to baseline

[2 Secondary Indexes (session + user)]
  Indexes Active:        2 | Duration: 1458.51 ms | Rate: 68,563 rows/sec
  Write Penalty:         4.22x relative to baseline

[4 Secondary Indexes (Full Coverage)]
  Indexes Active:        4 | Duration: 1738.09 ms | Rate: 57,534 rows/sec
  Write Penalty:         5.03x relative to baseline
------------------------------------------------------------
Secondary Index Write Penalty Verification: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_8> python scripts/run_all.py
======================================================================
DATA ENGINEERING LABORATORY — PRACTICAL 8 MASTER PIPELINE
Step 1: Multi-Million Clickstream Generation (2.5M rows)   [PASS] (31.51s)
Step 2: Linear Scan & Projection Overhead Diagnosis        [PASS] (0.38s)
Step 3: Secondary Indexing & ART Tree Optimization         [PASS] (29.86s)
Step 4: Physical Data Partitioning & Pruning               [PASS] (5.04s)
Step 5: Supplementary Nested Subquery Refactoring          [PASS] (18.95s)
Step 6: Secondary Index Write Penalty Benchmark            [PASS] (4.76s)
Step 7: Post-Laboratory Metrics & Comparison Report        [PASS] (0.06s)
----------------------------------------------------------------------
Total Pipeline Runtime: 90.57 seconds
ALL PRACTICAL 8 TASKS EXECUTED SUCCESSFULLY — STATUS: PASS
======================================================================
```

Final Conclusion

This practical demonstrates how query processing and optimization techniques improve the efficiency of analytical workloads. By creating a clickstream dataset and executing aggregation queries in DuckDB, the practical illustrates how execution plans can be inspected to identify operations such as table scans, filtering, aggregation, and sorting.

Indexing, filter pushdown, and query refactoring provide potential ways to improve query performance. Comparing execution times and query plans helps determine which techniques are effective for a particular workload.

Overall, the practical develops an understanding of database execution engines, analytical query optimization, and performance evaluation, which are important for building efficient and scalable data engineering systems.